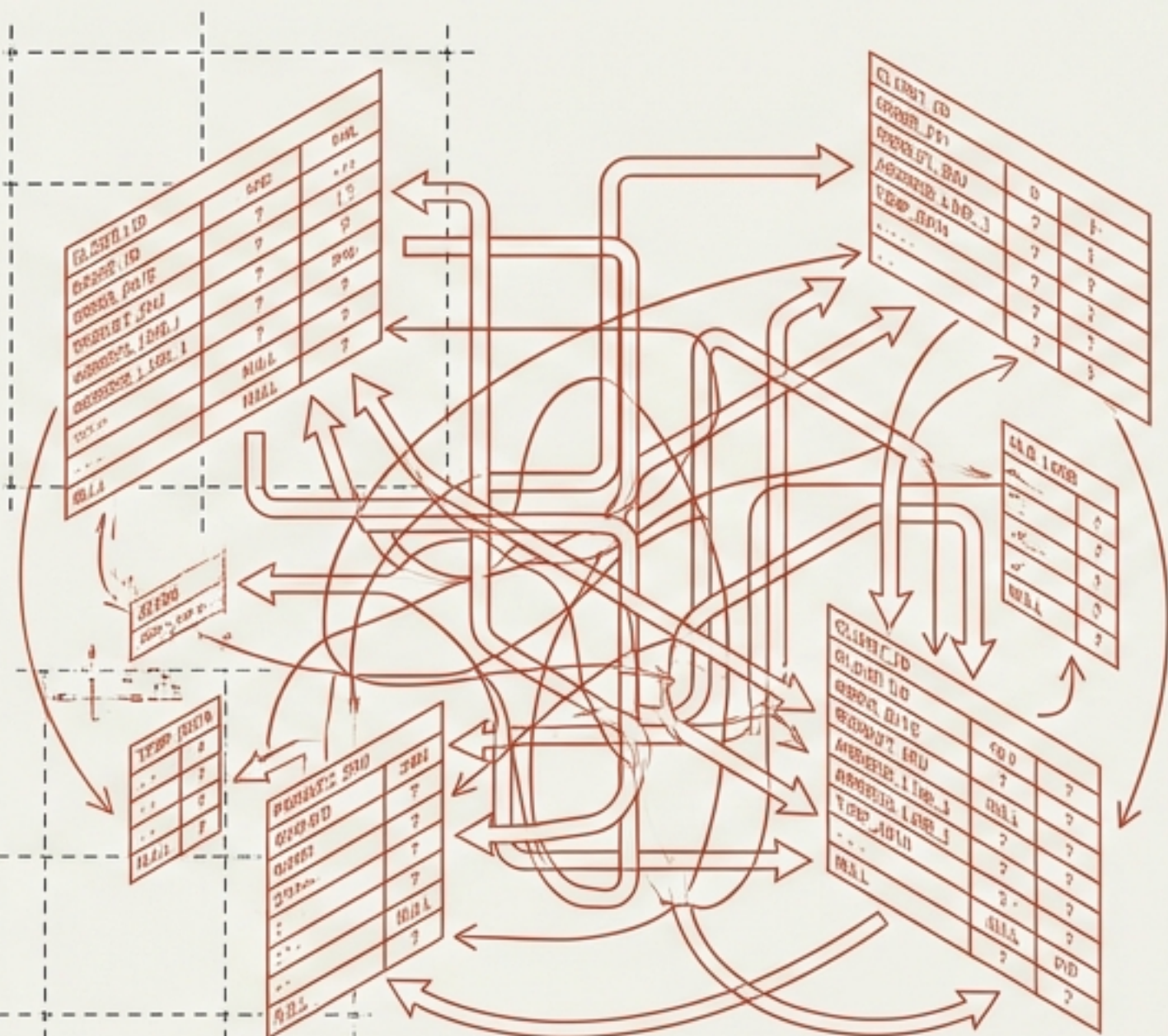


THE MODERN DATA BENCHMARK

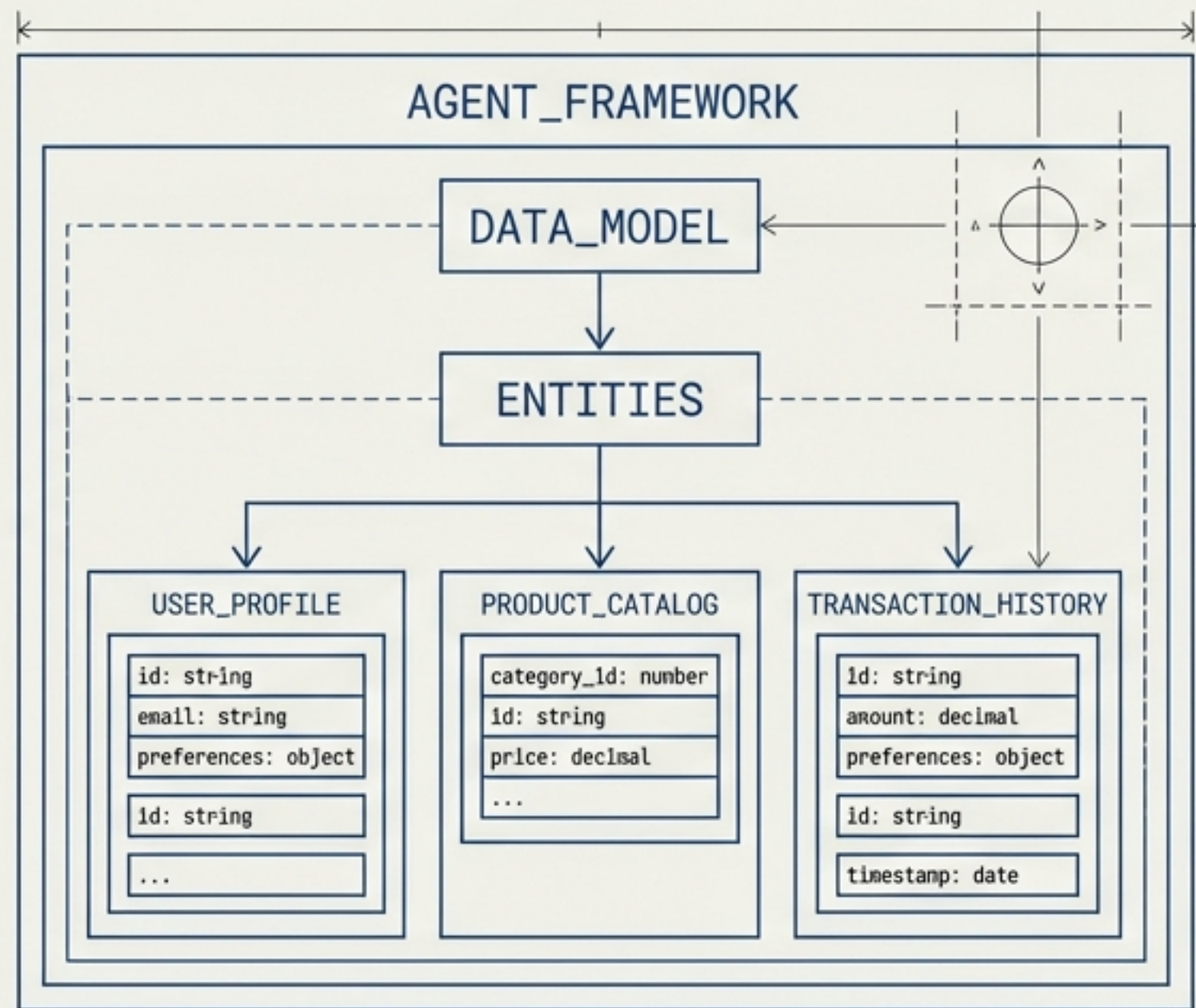
Why the Modern Data Stack (MDS) might be failing your AI Agents.



SQL Chaos
Brittle Schema

Manual Mapping
Data Silos

High Friction



TypeScript Structure
Type-Safe ORM

Automated Relations
Unified Access

Low Friction

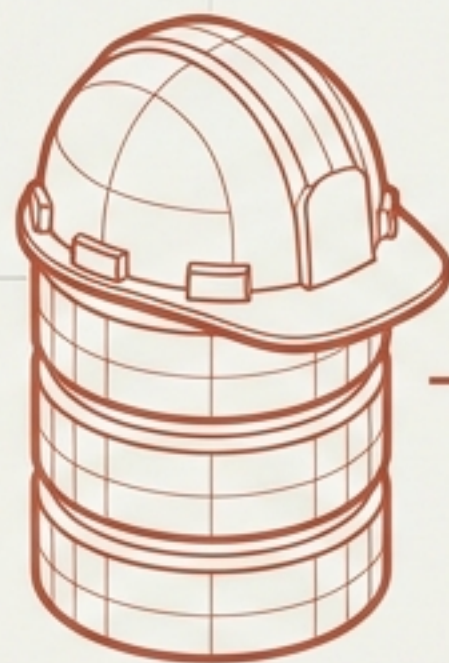
THE MDS SOLVED A HUMAN CONSTRAINT.

OPTIMIZED FOR HUMAN READABILITY



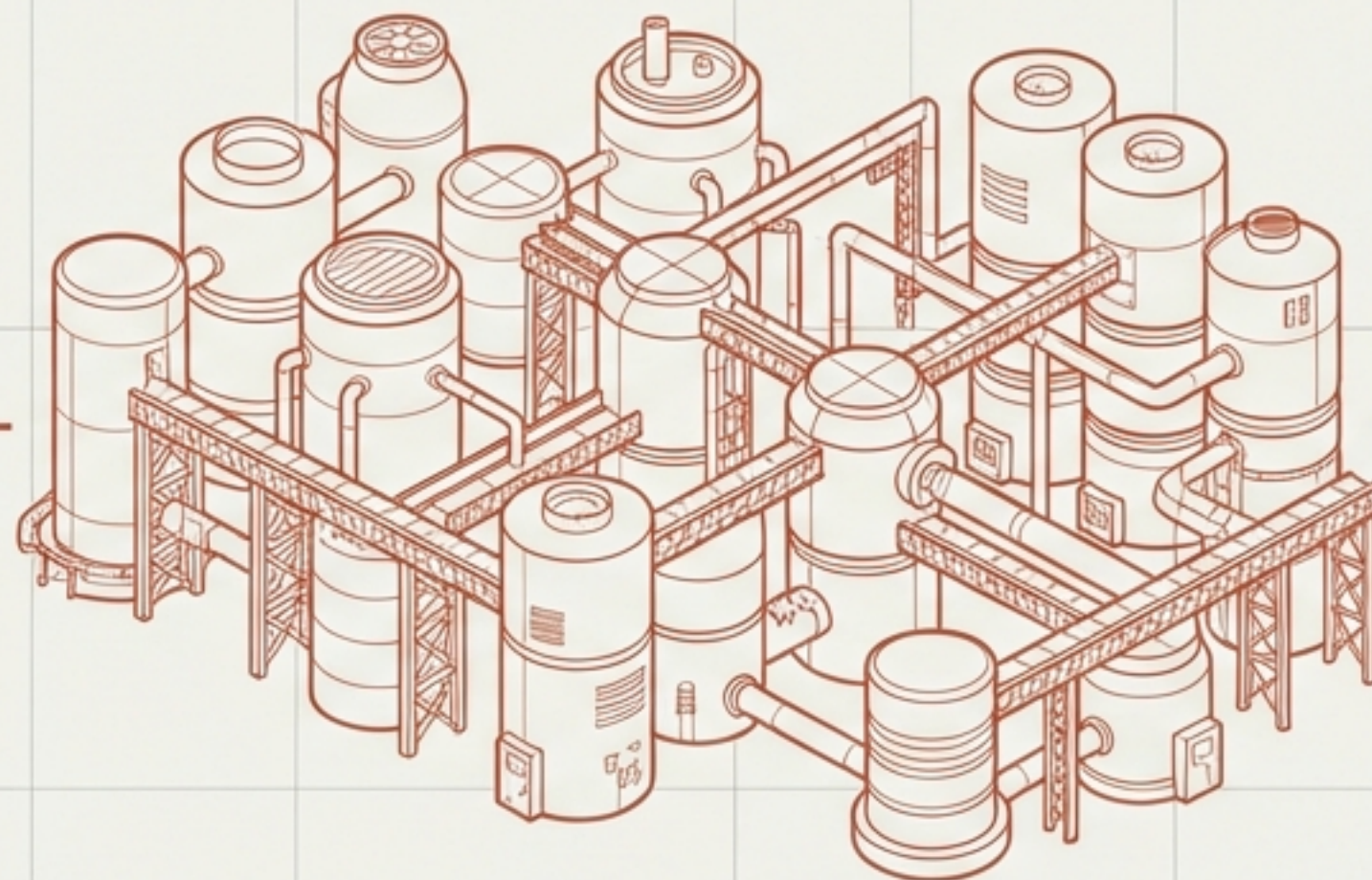
THE PROBLEM.

Analysts needed answers but couldn't write software.



THE SOLUTION (dbt).

SQL given engineering discipline.
Version control & templating.



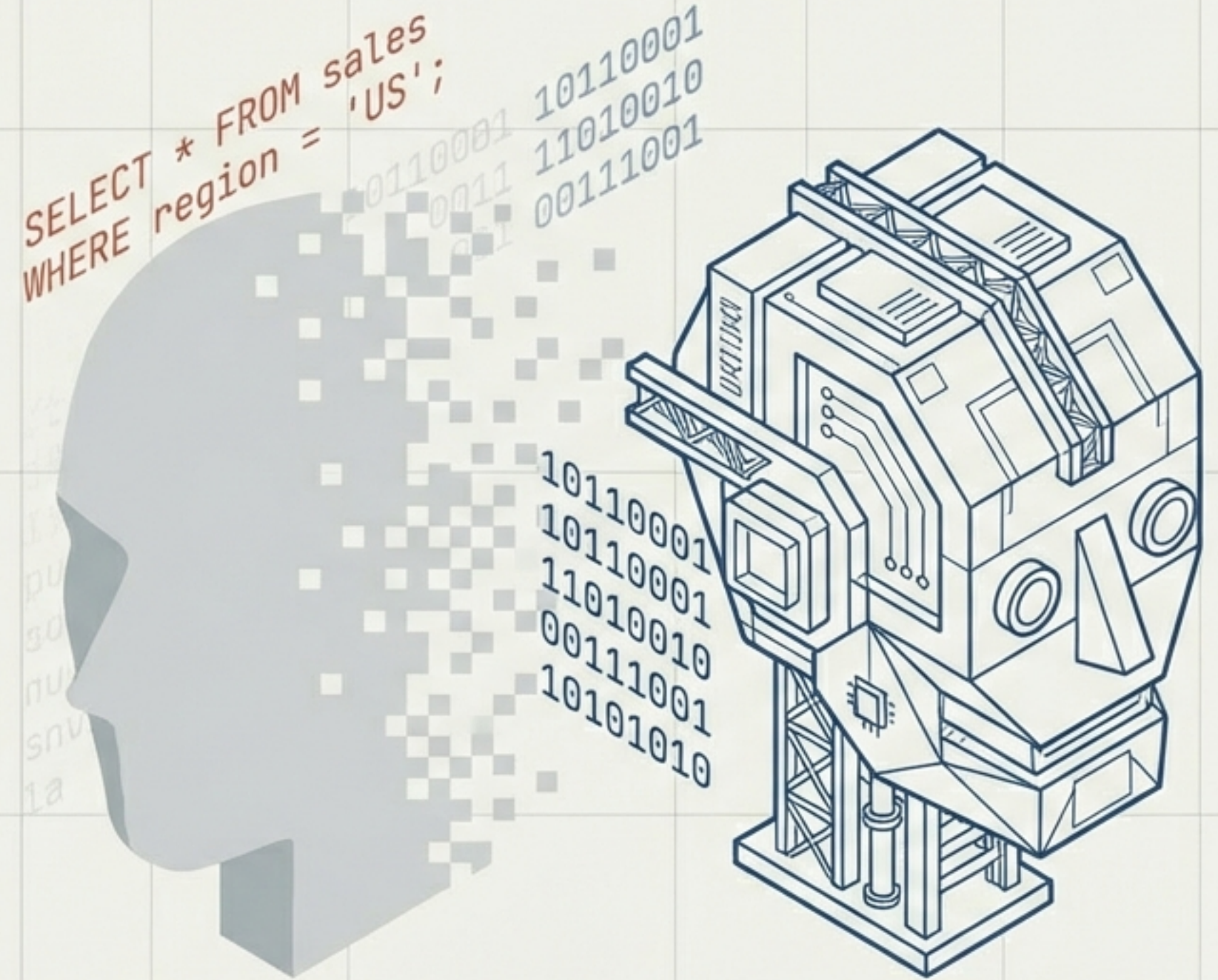
THE OUTCOME.

A massive ecosystem built on raw SQL.

THE CONSUMER OF DATA IS CHANGING.

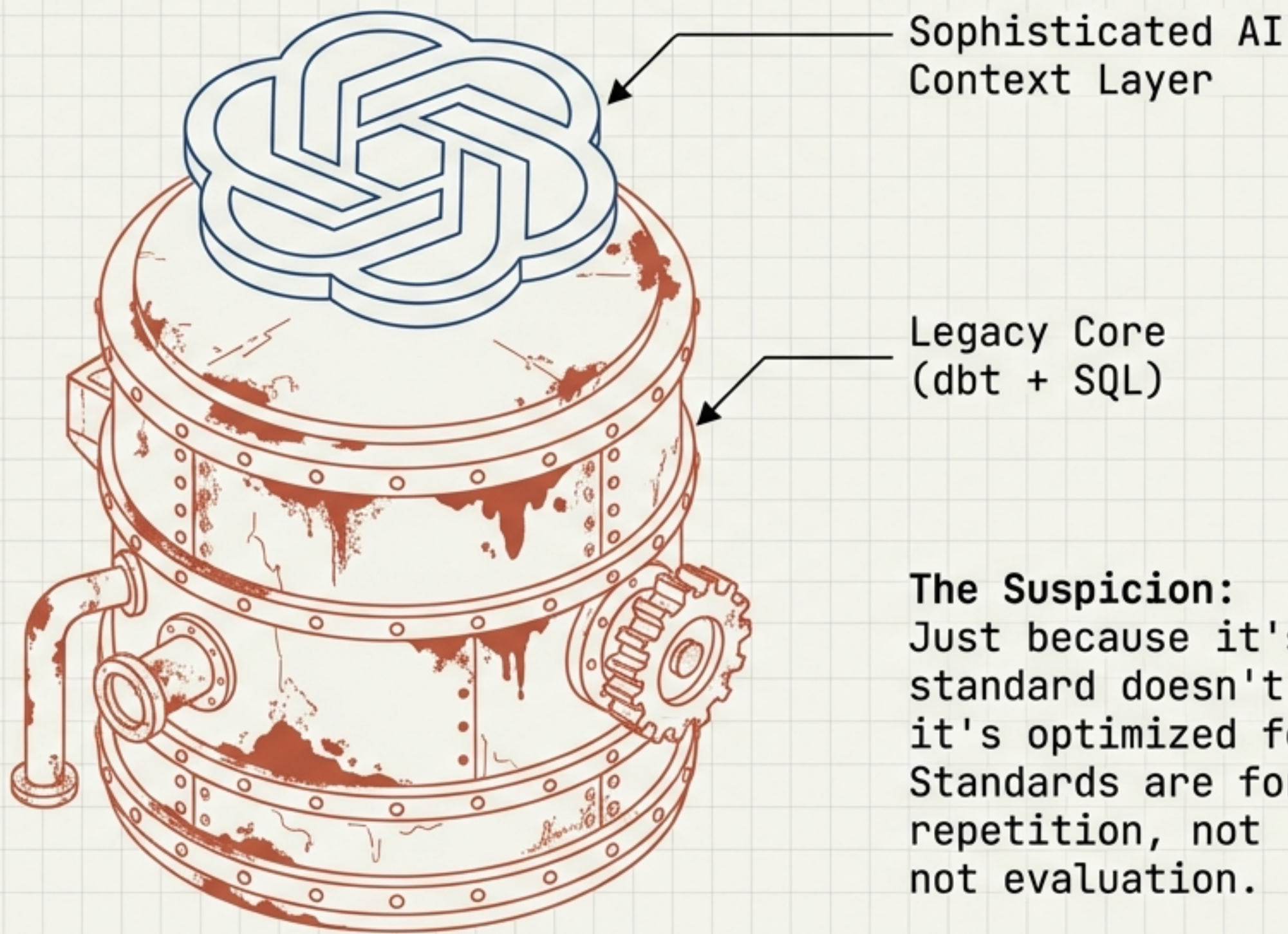
“If the consumer is an agent, ‘easiest for humans to read’ stops being important.

For an agent, which system is most legible, robust, and correct?



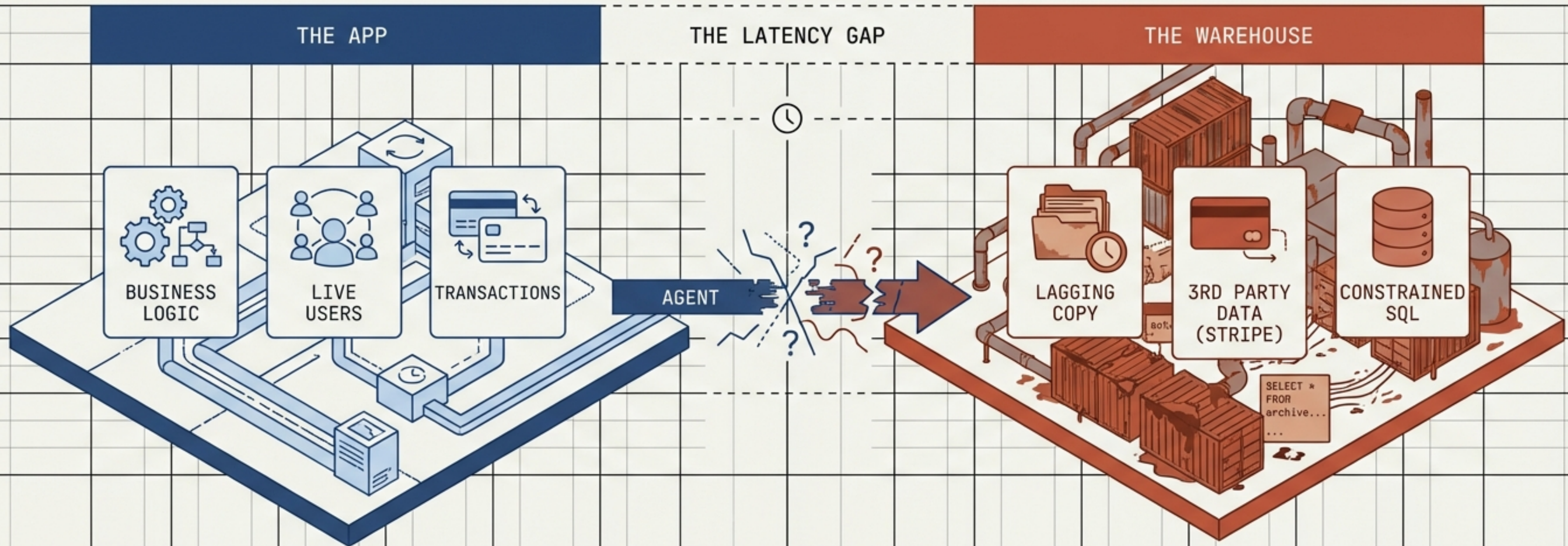
Even the frontier is stuck in the past.

Context:
OpenAI uses the "Covid-era MDS" driven by FinOps needs.



The Suspicion:
Just because it's standard doesn't mean it's optimized for LLMs. Standards are formed by repetition, not not evaluation.

THE ARCHITECTURAL DIVIDE.



The 'Source of Truth' in the warehouse is never truly trustworthy.

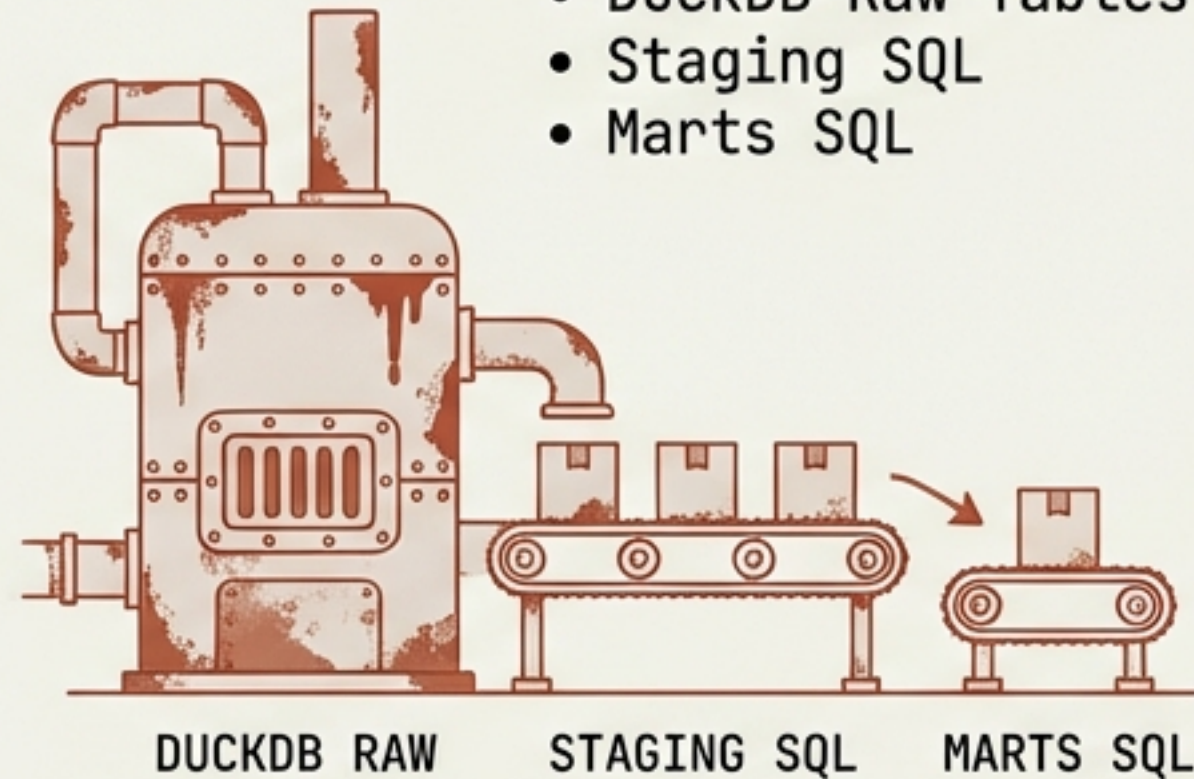
THE EXPERIMENT: MDS GYM.

THE METHODOLOGY

Goal: Read schema, write logic, return exact metric.
Tasks: ARPU, Churn, LTV. Rules: No hints. No hand-holding.

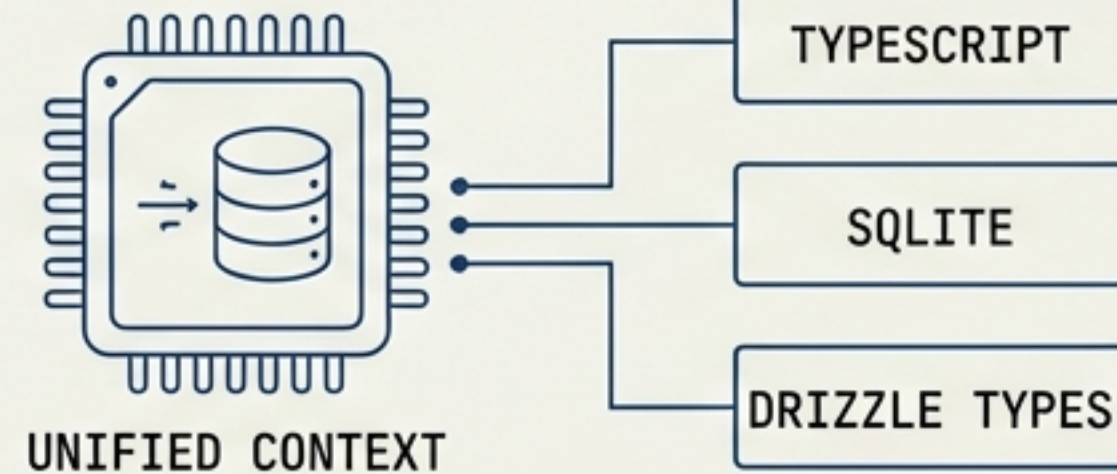
CONTENDER A: WAREHOUSE + DBT

- DuckDB Raw Tables
- Staging SQL
- Marts SQL



CONTENDER B: APP + DRIZZLE (ORM)

- Unified Context
- TypeScript
- SQLite
- Drizzle Types



SQL INDIRECTION VS. TYPED CONTEXT.

dbt / SQL

Guessing column names?
Weak typing.

```
SELECT * FROM staging_payments
LEFT JOIN users on
  staging_payments.user_id = users.id
WHERE payment_status = 'success'
ORDER BY payment_date DESC;
```

App / TypeScript

```
import { calculateARPU } from '@metrics'

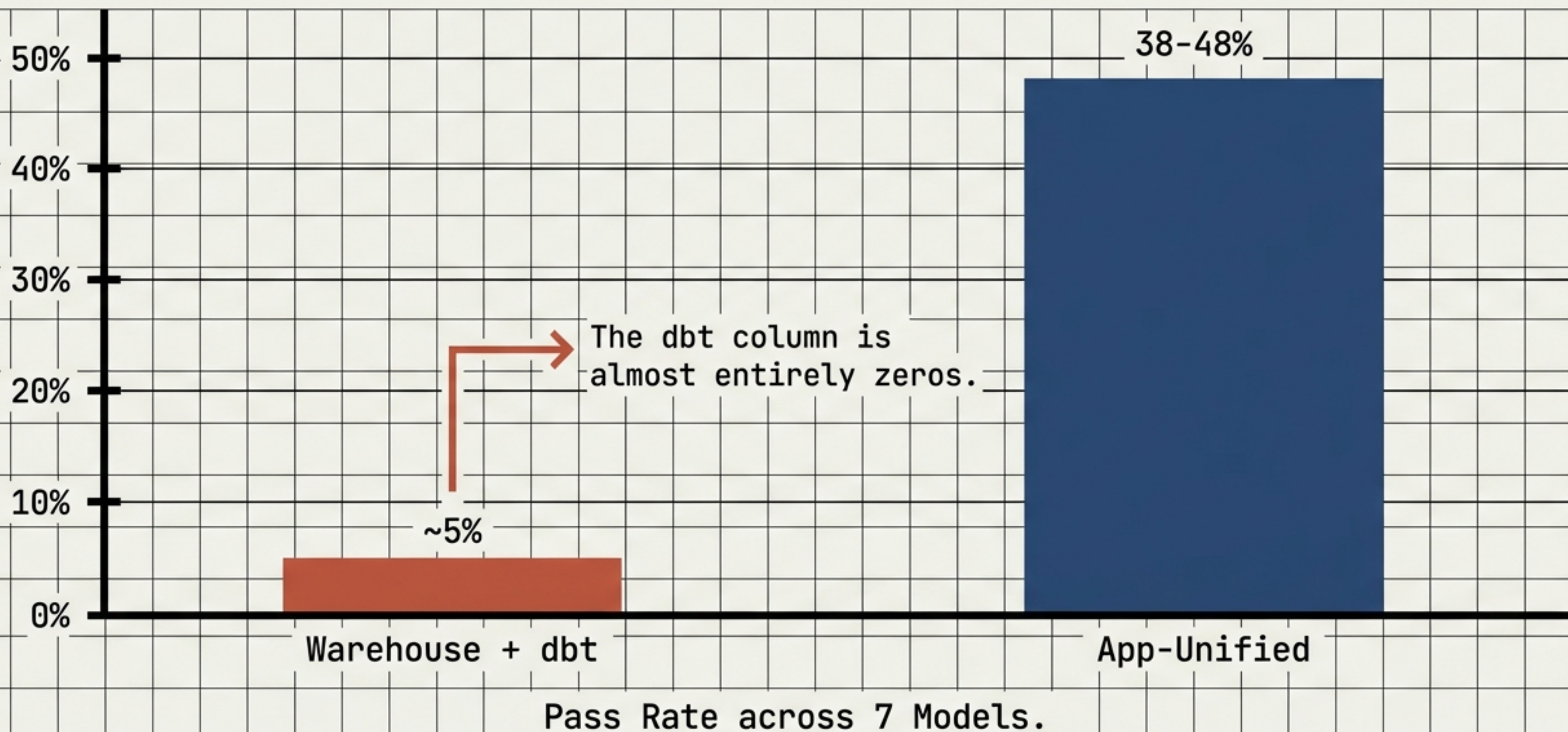
interface UserSchema {
  id: number;
  status: string;
}

const result: number =
  calculateARPU(currentUser: UserSchema)
```

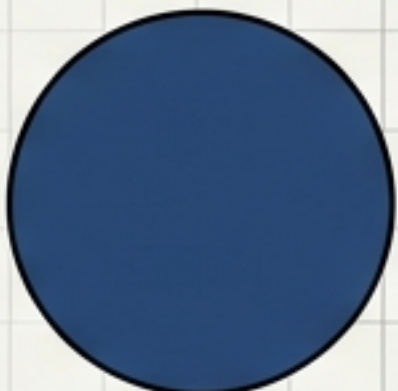
Type: UserSchema.
Confirmed.

In the App, the model has one typed context. In the Warehouse, it must guess.

ARCHITECTURE MATTERS MORE THAN THE MODEL.



EVEN THE SMARTEST MODELS STRUGGLE WITH SQL.

	ORM	dbt
Opus (Top Tier)	 3/3 Score	 Perfect on ORM. Failed 2/3 on dbt.
Haiku/Sonnet (Mid Tier)	 3/3 Score	 Wild fluctuation or total failure on dbt.

The architecture is what pushes them over or pulls them back.

ANATOMY OF FAILURE: WHERE AGENTS GET LOST.

Terminal Output

> SCHEMA MISMATCH ERROR.

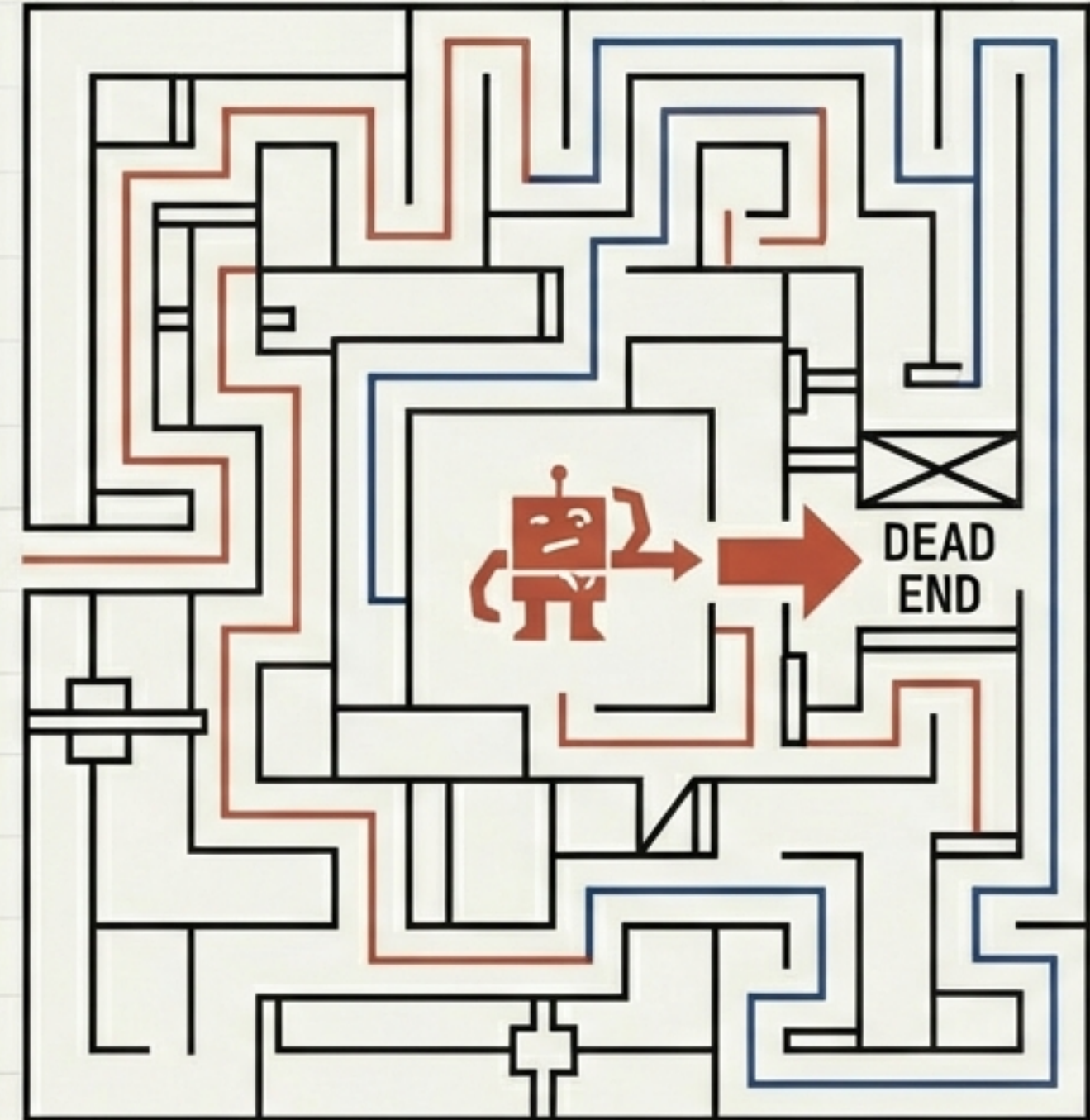
Input: 'created_at'.
Expected: 'usage_created_at'.
Agent was guessing.

> TYPE MISMATCH ERROR.

Operation: Interval Math.
Cause: Failed to cast VARCHAR
timestamp.

> INDIRECTION ERROR.

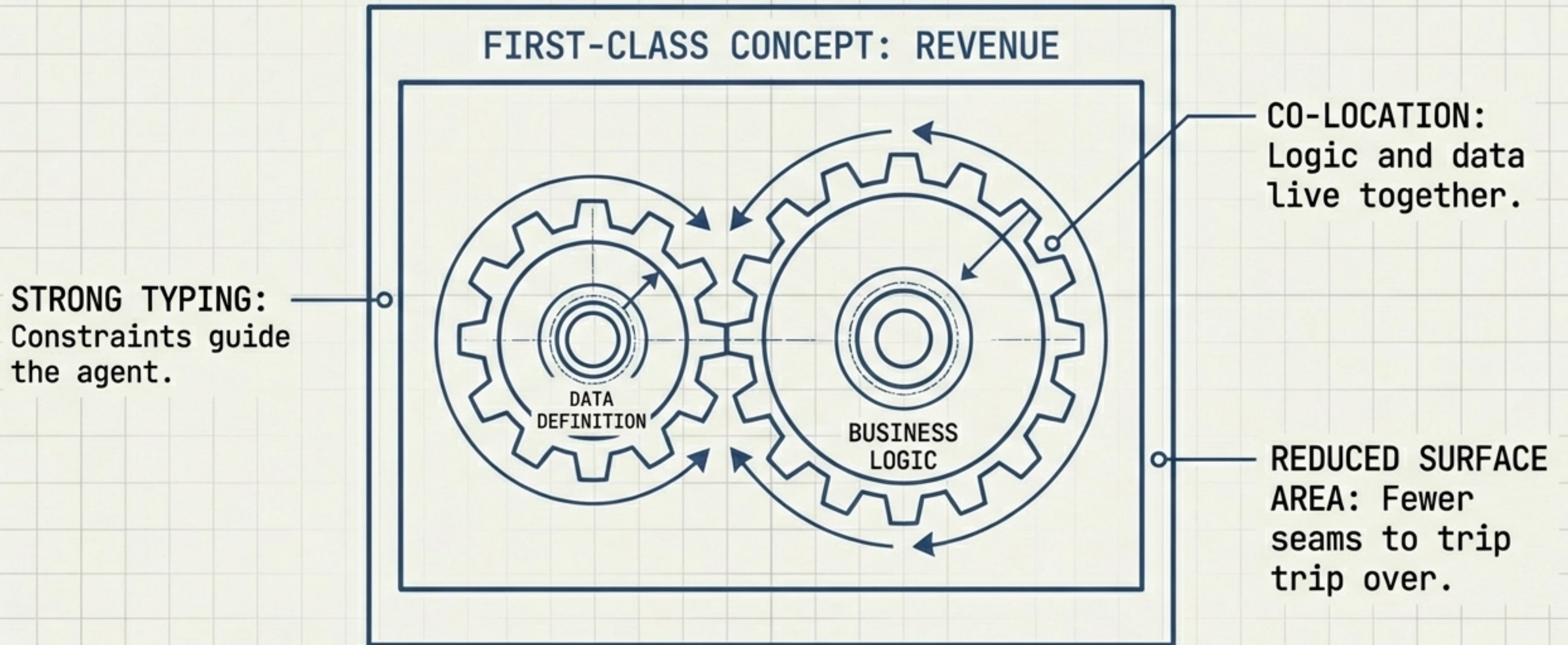
Agent timed out reading multiple
files (staging -> marts) to find
lineage.



The architecture is what pushes them over or pulls them back.

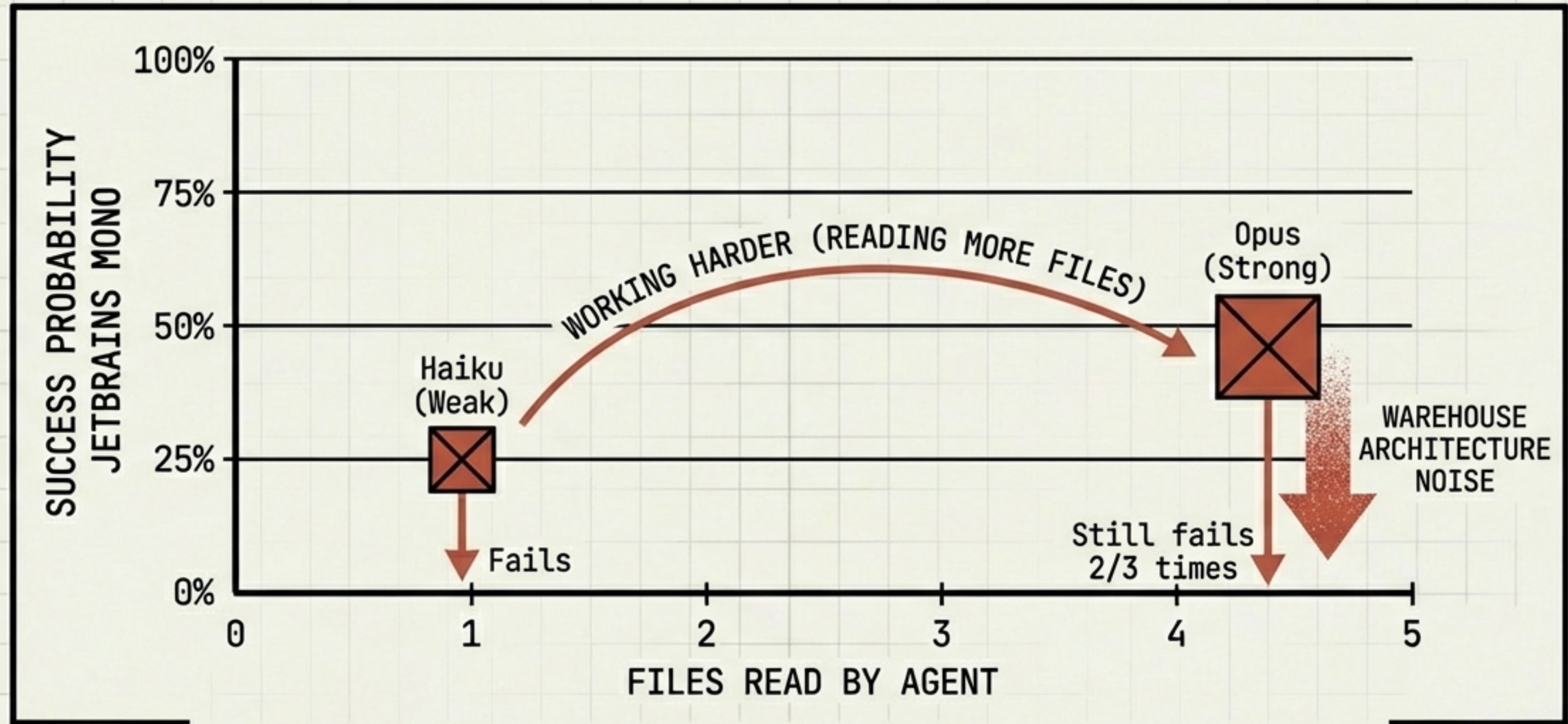


ANATOMY OF SUCCESS: WHY ORMs WON.

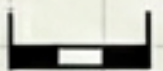


"Why not model revenue as a first-class concept?"

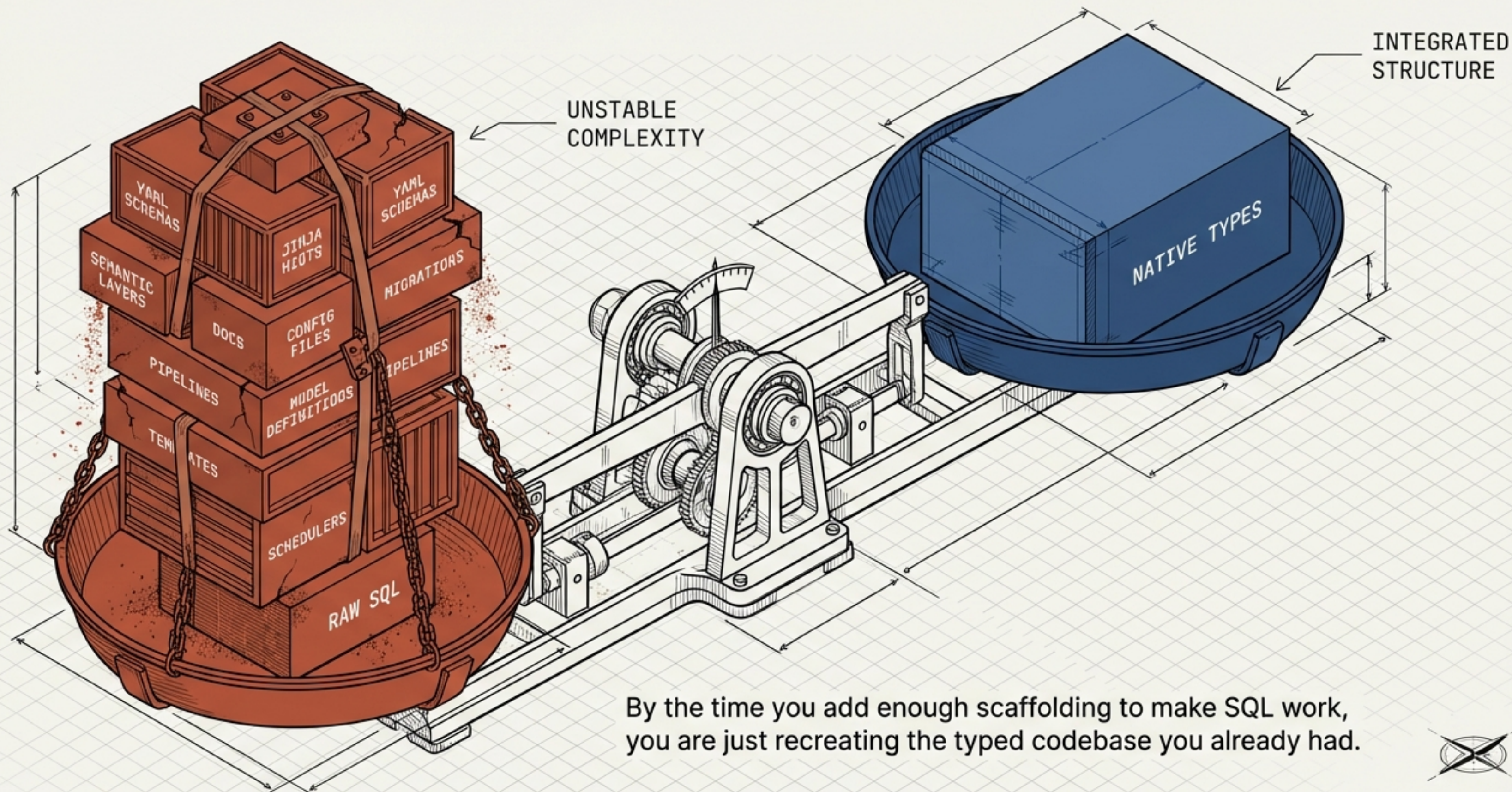
INDIRECTION HAS A COMPOUNDING COST.



Even when the model does its homework, the warehouse architecture introduces too much noise.



THE 'SCAFFOLDING' TRAP



By the time you add enough scaffolding to make SQL work, you are just recreating the typed codebase you already had.

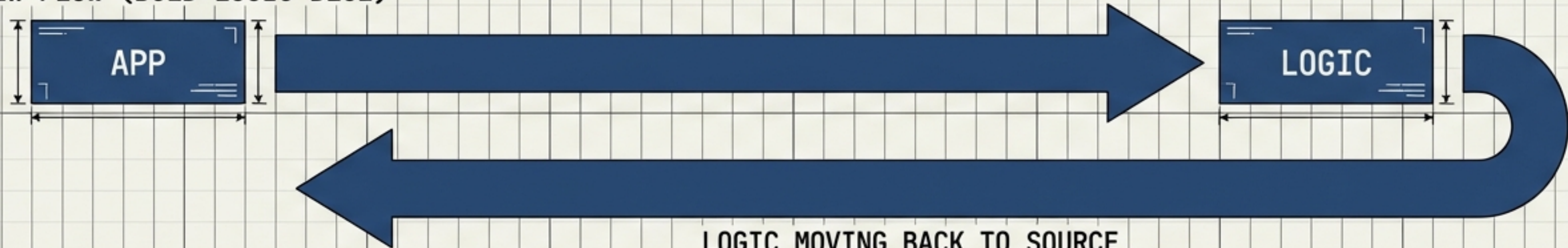


SHIFTING LEFT

TRADITIONAL FLOW (FADED)



NEW FLOW (BOLD LOGIC BLUE)



THE "CONTEXT LAYER" ISN'T A NEW TOOL;
IT'S YOUR APPLICATION CODE.

SPLIT-BRAIN GAP

STRUCTURAL INTEGRITY ESTABLISHED

SINGLE SOURCE

UNIFIED SOURCE OF TRUTH



SUMMARY OF FINDINGS



MDS was built for **humans**,
not **machines**.



SQL indirection **confuses**
agents; **Typed ORMs** empower them.



App-Unified is the **future**
for **Agentic Analytics**.



READ THE BENCHMARK.

Source: 'A Modern Data Benchmark'
by Matt Arderne (group by 1).

